

# 数学建模第三次作业

L<sup>A</sup>T<sub>E</sub>X by 常征 PB23030850

2026 年 5 月 10 日

## 目录

|          |                               |           |
|----------|-------------------------------|-----------|
| <b>1</b> | <b>问题建模</b>                   | <b>3</b>  |
| 1.1      | 任务描述 . . . . .                | 3         |
| 1.2      | 数学模型——监督学习的多分类问题 . . . . .    | 3         |
| 1.3      | 训练集与测试集的划分分析 . . . . .        | 3         |
| <b>2</b> | <b>神经网络的基本原理</b>              | <b>3</b>  |
| 2.1      | 神经元模型 . . . . .               | 3         |
| 2.2      | 多层感知机 . . . . .               | 3         |
| 2.3      | 激活函数 . . . . .                | 4         |
| 2.4      | 反向传播与梯度下降 . . . . .           | 4         |
| 2.5      | KNN (K-最近邻) . . . . .         | 4         |
| 2.6      | 随机森林 . . . . .                | 4         |
| <b>3</b> | <b>训练神经网络的基本流程</b>            | <b>5</b>  |
| <b>4</b> | <b>损失函数的数学表达式</b>             | <b>6</b>  |
| <b>5</b> | <b>评价指标</b>                   | <b>6</b>  |
| 5.1      | 严格数学定义 . . . . .              | 6         |
| 5.2      | 评价指标的实际意义 . . . . .           | 7         |
| <b>6</b> | <b>结果对比与分析</b>                | <b>7</b>  |
| 6.1      | 不同网络架构的对比 (数据集 1) . . . . .   | 7         |
| 6.2      | 不同激活函数的对比 (数据集 1) . . . . .   | 8         |
| 6.3      | Dropout 的影响 . . . . .         | 8         |
| 6.4      | 干净数据与含噪数据的对比 . . . . .        | 8         |
| 6.5      | 含噪数据集上的架构选择 . . . . .         | 9         |
| 6.6      | 分类报告 . . . . .                | 9         |
| 6.7      | 神经网络 vs KNN vs 随机森林 . . . . . | 9         |
| <b>7</b> | <b>结果可视化与分析</b>               | <b>11</b> |
| 7.1      | 数据分布 . . . . .                | 11        |
| 7.2      | 决策边界 . . . . .                | 12        |

|          |                          |           |
|----------|--------------------------|-----------|
| 7.3      | 含噪数据集的可视化 . . . . .      | 12        |
| 7.4      | 损失曲线分析 . . . . .         | 13        |
| 7.5      | KNN 与随机森林的决策边界 . . . . . | 13        |
| <b>8</b> | <b>总结与讨论</b>             | <b>14</b> |
| 8.1      | 实验结论 . . . . .           | 14        |
| 8.2      | AI 使用说明 . . . . .        | 14        |

# 1 问题建模

## 1.1 任务描述

本实验选择选项 2：使用神经网络对昆虫进行分类。已知数据集中的昆虫有 3 个类别（分别标记为 0, 1, 2），每个样本拥有两个特征：体长（ $x$ ）和翼长（ $y$ ）。实验提供两组数据集：数据集 1 为干净的测量数据，数据集 2 为带有测量噪声的数据。

## 1.2 数学模型——监督学习的多分类问题

将上述分类问题建模为**监督学习的多分类问题**。

设样本空间  $\mathcal{X} \subseteq \mathbb{R}^2$ ，每个样本  $\mathbf{x} = (x_1, x_2) \in \mathcal{X}$  分别表示体长和翼长。标签空间  $\mathcal{Y} = \{0, 1, 2\}$ ，表示三种昆虫类别。

给定训练集  $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^m$ ，其中  $y_i \in \mathcal{Y}$  为样本  $\mathbf{x}_i$  的真实类别标签。我们要学习一个分类函数  $f_\theta: \mathbb{R}^2 \rightarrow \mathbb{R}^3$ ，其输出为一个三维向量，第  $k$  个分量  $f_\theta(\mathbf{x})_k$  表示样本  $\mathbf{x}$  属于类别  $k$  的得分 (logit)。最终预测类别由下式确定：

$$\hat{y} = \arg \max_{k \in \{0, 1, 2\}} f_\theta(\mathbf{x})_k \quad (1)$$

训练目标为最小化训练集上的期望损失：

$$\theta^* = \arg \min_{\theta} \frac{1}{m} \sum_{i=1}^m \mathcal{L}(f_\theta(\mathbf{x}_i), y_i) \quad (2)$$

其中  $\mathcal{L}$  为损失函数（详见第 4 节）。

## 1.3 训练集与测试集的划分分析

实验要求分别在测试集的前 60 个数据和后 150 个数据上进行测试。前 60 个数据从训练集中随机抽取而来，其分布与训练集一致，用于验证模型是否充分拟合了训练数据；后 150 个数据为训练集之外的新数据，用于评估模型的**泛化能力**。若模型在前 60 个数据上准确率高而在后 150 个数据上准确率低，则说明模型存在过拟合。

# 2 神经网络的基本原理

## 2.1 神经元模型

人工神经网络的基本计算单元是神经元。一个神经元接收  $n$  个输入  $x_1, x_2, \dots, x_n$ ，通过加权求和并经过激活函数  $\sigma$  得到输出：

$$a = \sigma \left( \sum_{i=1}^n w_i x_i + b \right) \quad (3)$$

其中  $w_i$  为权重， $b$  为偏置项。

## 2.2 多层感知机

多层感知机由输入层、若干隐藏层和输出层组成。每一层的输出作为下一层的输入，通过逐层非线性变换，网络能够逼近任意复杂的函数关系。

设第  $\ell$  层的输出为  $\mathbf{h}^{(\ell)}$ :

$$\mathbf{h}^{(\ell)} = \sigma_{\ell}(\mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}) \quad (4)$$

其中  $\mathbf{W}^{(\ell)}$  为第  $\ell$  层的权重矩阵,  $\mathbf{b}^{(\ell)}$  为偏置向量,  $\sigma_{\ell}$  为激活函数。

### 2.3 激活函数

激活函数为网络引入非线性。常用的激活函数包括:

- **ReLU**:  $\sigma(x) = \max(0, x)$ , 计算简单, 缓解梯度消失;
- **Tanh**:  $\sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ , 输出以零为中心;
- **Sigmoid**:  $\sigma(x) = \frac{1}{1+e^{-x}}$ , 输出在  $(0, 1)$  之间;
- **Leaky ReLU**:  $\sigma(x) = \max(0.1x, x)$ , 解决 ReLU 的”死亡神经元”问题。

### 2.4 反向传播与梯度下降

神经网络的训练通过反向传播算法实现。前向传播计算预测值, 反向传播利用链式法则计算损失关于每个参数的梯度, 随后使用梯度下降法更新参数:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L} \quad (5)$$

其中  $\eta$  为学习率,  $\nabla_{\theta} \mathcal{L}$  为损失函数关于参数  $\theta$  的梯度。

### 2.5 KNN (K-最近邻)

KNN 是一种非参数的惰性学习方法。对于待分类样本  $\mathbf{x}$ , 在训练集中找到与其最近的  $k$  个样本, 根据多数投票决定类别。

两点距离使用欧氏距离:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{d=1}^2 (x_{i,d} - x_{j,d})^2} \quad (6)$$

预测类别的决策规则为多数投票:

$$\hat{y} = \arg \max_{c \in \{0, 1, 2\}} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbb{I}\{y_i = c\} \quad (7)$$

其中  $\mathcal{N}_k(\mathbf{x})$  为  $\mathbf{x}$  的  $k$  个最近邻构成的邻域。

$k$  是核心超参数:  $k$  过小, 模型对噪声敏感、方差大;  $k$  过大, 决策边界过于平滑、偏差大。

### 2.6 随机森林

随机森林是集成学习方法, 通过构造多棵决策树并投票得到最终预测。

每棵树由 Bootstrap 采样得到的子数据集训练。在每个分裂节点, 从  $p$  个特征中随机选取  $\sqrt{p}$  个候选特征, 选择最优分裂。分裂标准为**基尼不纯度**:

$$\text{Gini}(S) = 1 - \sum_{c=0}^2 \left( \frac{|S_c|}{|S|} \right)^2 \quad (8)$$

其中  $S$  为当前节点的样本集合,  $S_c$  为属于类别  $c$  的子集。基尼不纯度越小, 样本越纯。

最终预测由  $M$  棵决策树投票决定：

$$\hat{y} = \arg \max_c \sum_{m=1}^M \mathbb{I}\{h_m(\mathbf{x}) = c\} \quad (9)$$

Bootstrap 采样和随机特征选择的双重随机性降低了过拟合风险，使随机森林具有良好的泛化能力。

### 3 训练神经网络的基本流程

本实验训练神经网络的基本流程如下：

1. **数据预处理**：加载训练集与测试集数据，使用 StandardScaler 对特征进行标准化，使每个特征的均值为 0、方差为 1，以加速训练收敛。
2. **模型构建**：使用 PyTorch 搭建全连接神经网络。输入层为 2 个神经元（体长、翼长），若干隐藏层使用指定激活函数，输出层为 3 个神经元（对应 3 个类别）。
3. **前向传播**：将批次样本输入网络，逐层计算得到 logits 输出。
4. **损失计算**：使用交叉熵损失函数计算模型输出与真实标签之间的差异。
5. **反向传播**：调用 `loss.backward()` 自动计算损失关于所有可学习参数的梯度。
6. **参数更新**：使用 Adam 优化器根据梯度更新网络参数。
7. **学习率调整**：使用 ReduceLROnPlateau 调度器，当损失不再下降时自动降低学习率。
8. **迭代训练**：重复步骤 3-7 直至达到预设的训练轮数（epochs）。
9. **模型评估**：在测试集上计算分类准确率、混淆矩阵等指标。

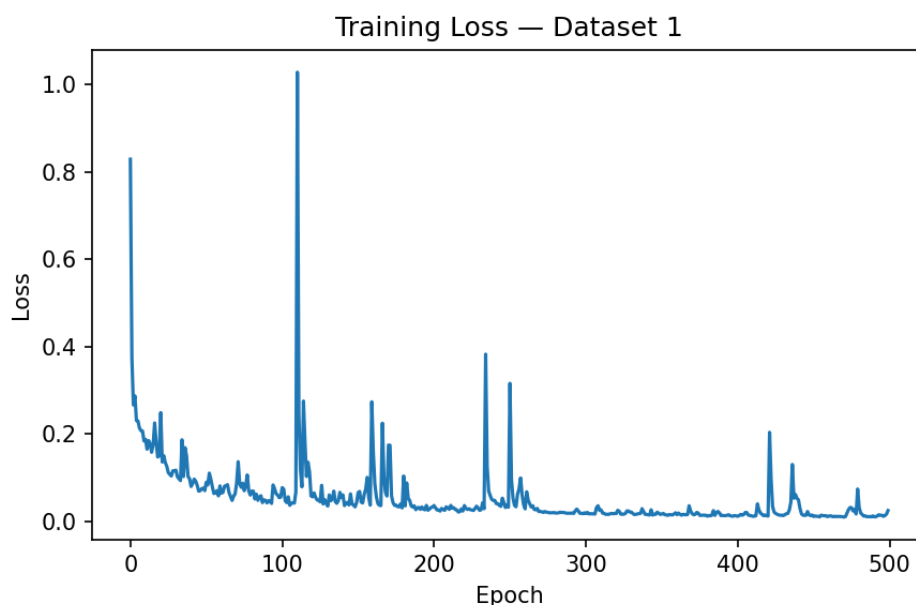


图 1: 数据集 1 上的训练损失曲线（架构：[32, 16], ReLU）

## 4 损失函数的数学表达式

本实验采用**交叉熵损失函数**。对于多分类问题，设样本总数为  $m$ ，类别数为  $C = 3$ 。首先对网络的输出 logits  $\mathbf{z}_i = f_\theta(\mathbf{x}_i) \in \mathbb{R}^3$  应用 Softmax 函数，得到预测概率分布：

$$\hat{p}_{i,k} = \frac{e^{z_{i,k}}}{\sum_{j=0}^2 e^{z_{i,j}}}, \quad k = 0, 1, 2 \quad (10)$$

交叉熵损失定义为真实标签分布与预测概率分布之间的交叉熵：

$$\mathcal{L}_{\text{CE}} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=0}^2 \mathbb{I}\{y_i = k\} \cdot \log \hat{p}_{i,k} \quad (11)$$

其中  $\mathbb{I}\{\cdot\}$  为示性函数，当条件成立时取 1，否则取 0。由于每个样本只属于一个类别，上式等价于：

$$\mathcal{L}_{\text{CE}} = -\frac{1}{m} \sum_{i=1}^m \log \hat{p}_{i,y_i} \quad (12)$$

直观上，当模型对真实类别的预测概率  $\hat{p}_{i,y_i}$  越接近 1 时，损失越小；反之，预测概率越接近 0 时损失趋于无穷大，形成强烈的惩罚信号。

## 5 评价指标

### 5.1 严格数学定义

#### 准确率 (Accuracy)

准确率定义为分类正确的样本数占样本总数的比例：

$$\text{Accuracy} = \frac{1}{m} \sum_{i=1}^m \mathbb{I}\{\hat{y}_i = y_i\} \quad (13)$$

其中  $\hat{y}_i = \arg \max_k f_\theta(\mathbf{x}_i)_k$  为预测类别， $y_i$  为真实类别。准确率是最直观的分类性能度量，取值范围为  $[0, 1]$ 。

#### 精确率 (Precision)

对于类别  $k$ ，精确率定义为预测为类别  $k$  的样本中真正属于类别  $k$  的比例：

$$\text{Precision}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k} \quad (14)$$

其中  $\text{TP}_k$  为真正例（被正确分类为类别  $k$  的样本数）， $\text{FP}_k$  为假正例（被错误分类为类别  $k$  的样本数）。

#### 召回率 (Recall)

对于类别  $k$ ，召回率定义为真实类别  $k$  的样本中被正确识别的比例：

$$\text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k} \quad (15)$$

其中  $\text{FN}_k$  为假负例（真实类别为  $k$  但被预测为其他类别的样本数）。

## F1 分数 (F1-Score)

F1 分数是精确率和召回率的调和平均:

$$F1_k = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \quad (16)$$

## 5.2 评价指标的实际意义

- **准确率**反映了模型的整体分类能力。在本实验中，它直接回答了”模型能正确分类多少比例的昆虫”这一核心问题。
- **精确率**关注预测结果的可靠性。当精确率低时，意味着模型将其他类别的昆虫误报为当前类别。在害虫防治场景中，若将益虫误判为害虫会导致不必要的农药使用。
- **召回率**关注某一类别是否被遗漏。若某种害虫的召回率低，说明大量该害虫未被识别，可能导致虫害扩散。
- **F1 分数**在精确率和召回率之间取得平衡，适用于类别分布不均衡或对误报、漏报都敏感的场景。

# 6 结果对比与分析

## 6.1 不同网络架构的对比 (数据集 1)

在干净数据集上固定激活函数为 ReLU，测试不同隐藏层配置对分类准确率的影响。

表 1: 不同网络架构的测试准确率对比 (数据集 1, ReLU)

| 架构          | 训练准确率  | 测试 (全部)       | 测试 (后 150)    |
|-------------|--------|---------------|---------------|
| [8]         | 0.9867 | 0.9714        | 0.9667        |
| [16]        | 0.9844 | 0.9714        | 0.9733        |
| [32]        | 0.9889 | 0.9762        | 0.9800        |
| [64]        | 0.9911 | 0.9857        | 0.9800        |
| [16, 8]     | 0.9844 | 0.9667        | 0.9600        |
| [32, 16]    | 0.9844 | 0.9762        | 0.9733        |
| [64, 32]    | 0.9956 | <b>1.0000</b> | <b>1.0000</b> |
| [128, 64]   | 0.9978 | <b>1.0000</b> | <b>1.0000</b> |
| [32, 16, 8] | 0.9978 | 0.9952        | 0.9933        |

分析: 双隐藏层结构 [64, 32] 和 [128, 64] 在测试集上达到了 100% 的准确率, 说明该架构能够充分拟合数据的非线性决策边界。单隐藏层结构表现稍差, 特别是仅 8 个神经元时模型容量不足。三层隐藏层 [32, 16, 8] 反而略低于双层结构, 说明对于此简单任务, 过深的网络可能导致过拟合。

## 6.2 不同激活函数的对比（数据集 1）

表 2: 不同激活函数的测试准确率对比（数据集 1，架构 [32, 16]）

| 激活函数       | 训练准确率  | 测试（全部）        | 测试（后 150）     |
|------------|--------|---------------|---------------|
| ReLU       | 0.9844 | 0.9762        | 0.9733        |
| Tanh       | 0.9933 | 0.9952        | 0.9933        |
| Sigmoid    | 0.9800 | 0.9857        | 0.9800        |
| Leaky ReLU | 0.9956 | <b>1.0000</b> | <b>1.0000</b> |

分析：Leaky ReLU 达到 100% 准确率，优于普通 ReLU。这是因为 Leaky ReLU 在负半轴保留了小的梯度，避免了神经元“死亡”问题。Tanh 也表现优秀（99.52%），其零中心输出有助于梯度传播。Sigmoid 收敛稍慢但最终准确率仍达 98.57%。

## 6.3 Dropout 的影响

表 3: 不同 Dropout 比例的影响（数据集 1，架构 [64, 32]，ReLU）

| Dropout | 训练准确率  | 测试（全部）        | 测试（后 150）     |
|---------|--------|---------------|---------------|
| 0.0     | 0.9956 | <b>1.0000</b> | <b>1.0000</b> |
| 0.1     | 0.9956 | 0.9857        | 0.9800        |
| 0.3     | 0.9933 | 0.9810        | 0.9733        |
| 0.5     | 0.9844 | 0.9667        | 0.9600        |

分析：Dropout 在此任务中反而降低了准确率。原因是该任务较为简单（二维特征、三类分类、线性可分的趋势明显），Dropout 的随机丢弃削弱了网络的表达能力。Dropout 通常在更复杂、参数量更大的任务上才能发挥正则化优势。

## 6.4 干净数据与含噪数据的对比

表 4: 数据集 1（干净）与数据集 2（含噪）的对比（架构 [32, 16]，ReLU）

| 数据集       | 训练准确率  | 测试（全部） | 测试（前 60） | 测试（后 150） |
|-----------|--------|--------|----------|-----------|
| 数据集 1（干净） | 0.9844 | 0.9762 | 0.9833   | 0.9733    |
| 数据集 2（含噪） | 0.9111 | 0.9143 | 0.8500   | 0.9400    |

分析：数据集 2 由于含有测量噪声，分类准确率明显低于数据集 1。值得注意的现象是：数据集 2 上测试前 60 个样本的准确率（85%）显著低于后 150 个样本（94%）。这是因为数据集 2 的前 60 个测试样本同样来自带有噪声的训练集，噪声导致标签本身不可靠；而后 150 个样本是全新的，模型反而能更好地泛化。这说明含噪标签是影响模型性能的主要因素，而非模型的泛化能力不足。



## 6.5 含噪数据集上的架构选择

表 5: 含噪数据集上不同架构的对比 (数据集 2, ReLU)

| 架构       | 训练准确率  | 测试 (全部)       | 测试 (后 150)    |
|----------|--------|---------------|---------------|
| [16]     | 0.8911 | <b>0.9381</b> | <b>0.9467</b> |
| [32, 16] | 0.9111 | 0.9143        | 0.9400        |
| [64, 32] | 0.9178 | <b>0.9381</b> | <b>0.9467</b> |

分析: 在含噪数据上, 最简单的单隐藏层架构 [16] 反而取得了与 [64, 32] 相同的最优泛化性能。这说明面对标签噪声, 较小的模型容量有助于避免过度拟合噪声标签, 起到了隐式的正则化作用。

## 6.6 分类报告

数据集 1 (架构 [64, 32], Leaky ReLU) 在测试集上的详细分类报告:

表 6: 数据集 1 的分类报告

| 类别       | 精确率        | 召回率  | F1 分数 | 样本数 |
|----------|------------|------|-------|-----|
| Class 0  | 1.00       | 1.00 | 1.00  | 70  |
| Class 1  | 1.00       | 1.00 | 1.00  | 70  |
| Class 2  | 1.00       | 1.00 | 1.00  | 70  |
| Accuracy | 1.00 (210) |      |       |     |

数据集 2 (架构 [64, 32], ReLU) 在测试集上的详细分类报告:

表 7: 数据集 2 的分类报告

| 类别       | 精确率        | 召回率  | F1 分数 | 样本数 |
|----------|------------|------|-------|-----|
| Class 0  | 0.88       | 0.96 | 0.92  | 70  |
| Class 1  | 1.00       | 0.83 | 0.91  | 70  |
| Class 2  | 0.88       | 0.96 | 0.92  | 70  |
| Accuracy | 0.91 (210) |      |       |     |

从表 7 可以看出, 类别 1 的召回率最低 (0.83), 表明约有 17% 的真实类别 1 样本被误分类。类别 1 的精确率最高 (1.00), 表明没有其他类别的样本被误判为类别 1。

## 6.7 神经网络 vs KNN vs 随机森林

为进一步验证神经网络方法的有效性, 本实验引入 KNN 和随机森林作为基准方法进行对比。

## KNN 结果

表 8: KNN 在不同  $k$  值下的分类准确率

| 数据集 | $k$ | 训练准确率  | 测试 (全部)       | 测试 (后 150)    |
|-----|-----|--------|---------------|---------------|
| 1   | 1   | 1.0000 | 0.9810        | 0.9733        |
| 1   | 3   | 0.9756 | 0.9619        | 0.9533        |
| 1   | 5   | 0.9622 | 0.9571        | 0.9533        |
| 1   | 7   | 0.9644 | 0.9429        | 0.9333        |
| 1   | 9   | 0.9622 | 0.9429        | 0.9400        |
| 2   | 1   | 1.0000 | <b>0.9667</b> | <b>0.9533</b> |
| 2   | 3   | 0.9200 | 0.9190        | 0.9267        |
| 2   | 5   | 0.9089 | 0.9286        | 0.9400        |
| 2   | 7   | 0.9022 | 0.9095        | 0.9200        |
| 2   | 9   | 0.8978 | 0.9048        | 0.9133        |

KNN( $k = 1$ ) 在数据集 1 上达到 98.10%，但低于神经网络的 100%。在含噪数据集 2 上，KNN( $k = 1$ ) 达到 96.67%，显著优于神经网络的 93.81%。这表明基于距离的局部投票对含噪标签有天然的鲁棒性—— $k = 1$  时模型“信任”每个邻居的标签，而标签虽然有测量噪声但并未被大规模错误标注。随  $k$  增大，准确率下降，因为更大的邻域引入了更多来自其他类别的干扰样本。

## 随机森林结果

表 9: 随机森林在不同参数下的分类准确率

| 数据集 | 参数                       | 训练准确率  | 测试 (全部)       | 测试 (后 150)    |
|-----|--------------------------|--------|---------------|---------------|
| 1   | $n=50$                   | 1.0000 | 0.9667        | 0.9533        |
| 1   | $n=100$                  | 1.0000 | 0.9619        | 0.9467        |
| 1   | $n=200$                  | 1.0000 | 0.9667        | 0.9533        |
| 1   | $n=100, \text{depth}=5$  | 0.9822 | 0.9476        | 0.9267        |
| 1   | $n=100, \text{depth}=10$ | 1.0000 | 0.9571        | 0.9400        |
| 2   | $n=50$                   | 0.9978 | <b>0.9571</b> | <b>0.9400</b> |
| 2   | $n=100$                  | 1.0000 | <b>0.9619</b> | <b>0.9467</b> |
| 2   | $n=200$                  | 1.0000 | <b>0.9619</b> | <b>0.9467</b> |
| 2   | $n=100, \text{depth}=5$  | 0.9311 | 0.9000        | 0.9067        |
| 2   | $n=100, \text{depth}=10$ | 1.0000 | 0.9571        | 0.9400        |

随机森林在数据集 1 上准确率约 96.7%，低于神经网络。限制树深度 ( $\text{depth}=5$ ) 时性能显著下降，说明此数据需要足够的模型复杂度。在含噪数据集 2 上，RF( $n=100$ ) 达到 96.19%，同样优于神经网络。

三种方法对比汇总

表 10: 三种方法的最优测试准确率对比

| 方法            | 数据集 1 (干净)    | 数据集 2 (含噪)    | 特点          |
|---------------|---------------|---------------|-------------|
| 神经网络          | <b>1.0000</b> | 0.9381        | 非线性拟合强，调参灵活 |
| KNN ( $k=1$ ) | 0.9810        | <b>0.9667</b> | 无需训练，对噪声鲁棒  |
| 随机森林          | 0.9667        | 0.9619        | 集成学习，稳定性好   |

**分析：**神经网络在干净数据上表现最优，但含噪数据上反而不如 KNN 和随机森林。原因有二：其一，神经网络通过梯度下降优化，容易过度拟合训练集中的噪声标签；其二，KNN( $k = 1$ )和随机森林的决策机制对个别噪声标签有更好的容错性——KNN 依赖近邻一致性，随机森林通过 Bootstrap 和随机特征选择减少了噪声的干扰。这一发现揭示了在实际应用中，当数据标注质量无法保证时，传统机器学习方法可能是更稳健的选择。

7 结果可视化与分析

7.1 数据分布

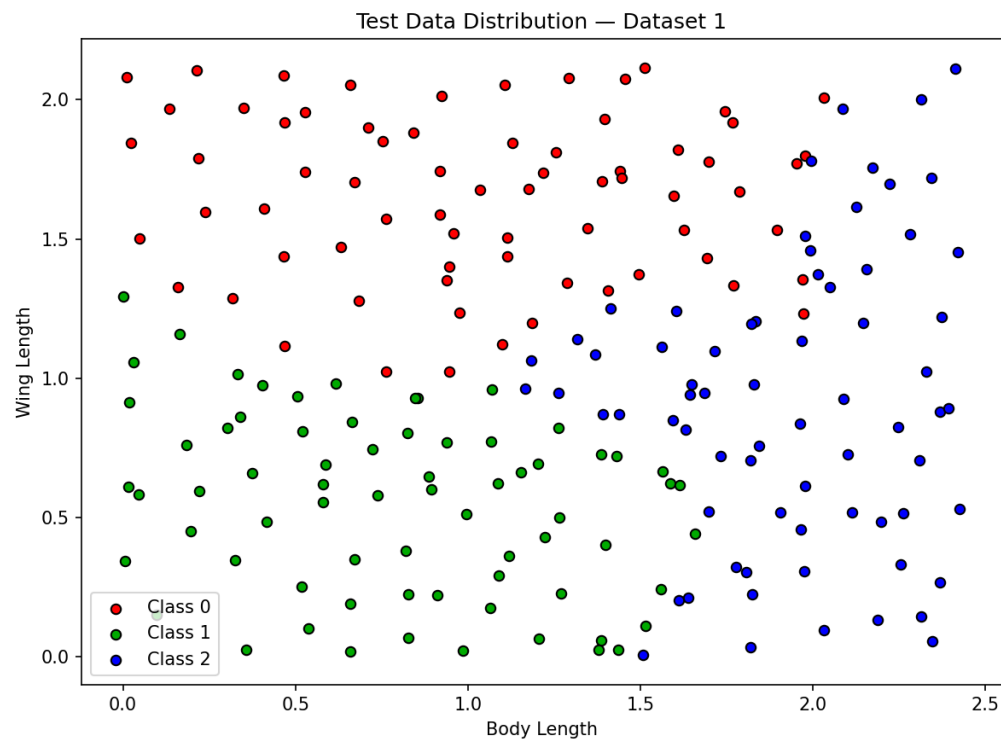


图 2: 数据集 1 测试数据的散点分布图

从图 2 可以看出，三类昆虫在体长-翼长平面上具有明显的聚类结构：类别 0 位于右上区域（大体长、大翼长），类别 1 位于左下区域（小体长、小翼长），类别 2 位于右下区域（大体长、小翼长）。这种线性可分的趋势解释了为何简单的全连接网络即可达到很高的分类准确率。

## 7.2 决策边界

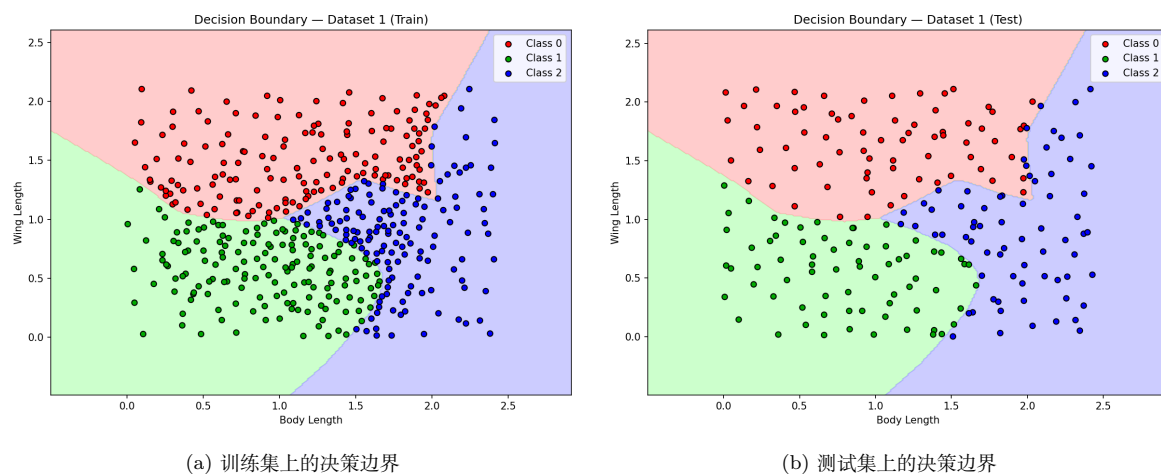


图 3: 数据集 1 上的决策边界 (架构: [32, 16], ReLU)

从图 3 可以看出, 神经网络学习到的决策边界是平滑的非线性曲线。类别 0 与类别 2 之间的边界较为复杂, 反映了这两个类别在特征空间中的重叠。决策边界在训练集和测试集上的形状基本一致, 说明模型泛化良好。

## 7.3 含噪数据集的可视化

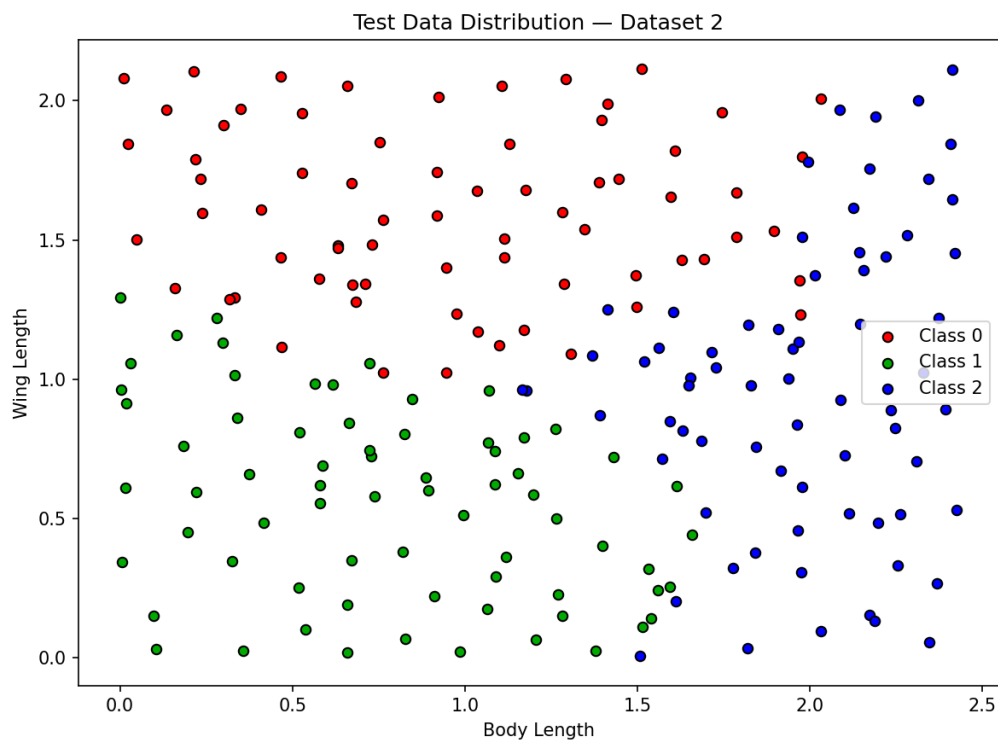


图 4: 数据集 2 (含噪) 测试数据的散点分布图

对比图 2 和图 4 可以看到，数据集 2 中类别间的边界更加模糊，存在明显的交叉区域。这与第 6 节中数据集 2 准确率下降的结论一致——测量噪声增加了类别间的混淆程度。

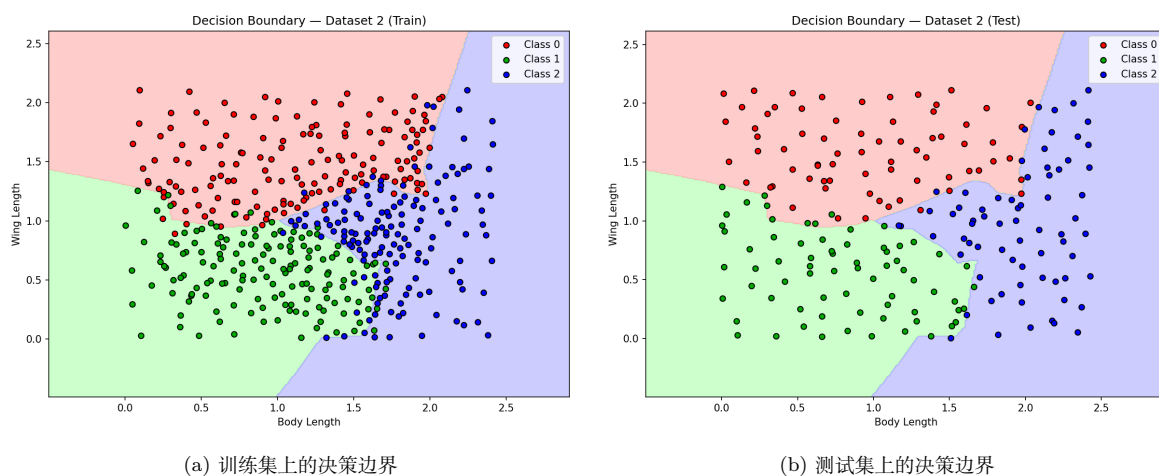


图 5: 数据集 2 (含噪) 上的决策边界 (架构: [32, 16], ReLU)

从图 5 可以看出，含噪数据上的决策边界更加曲折，模型试图拟合那些可能由噪声引起的异常点。但总体上决策边界的形状仍保持合理，说明模型具有一定的抗噪能力。

## 7.4 损失曲线分析

图 1 展示了训练过程中的损失下降曲线。损失在初期快速下降，随后逐步趋于平滑，在约 200 轮后基本收敛。没有出现损失上升或剧烈震荡的现象，说明学习率设置合理，模型训练稳定。

## 7.5 KNN 与随机森林的决策边界

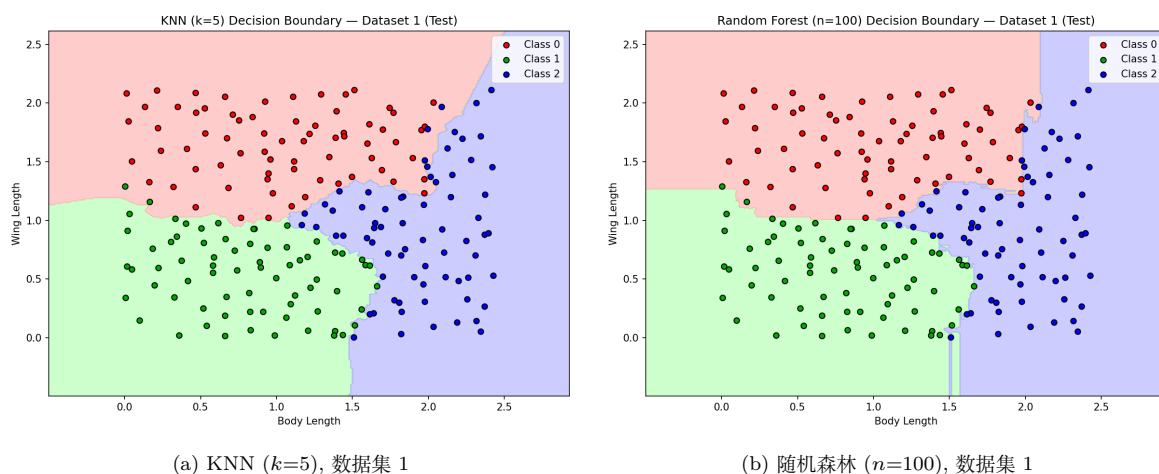


图 6: KNN 与随机森林在数据集 1 (干净) 上的决策边界

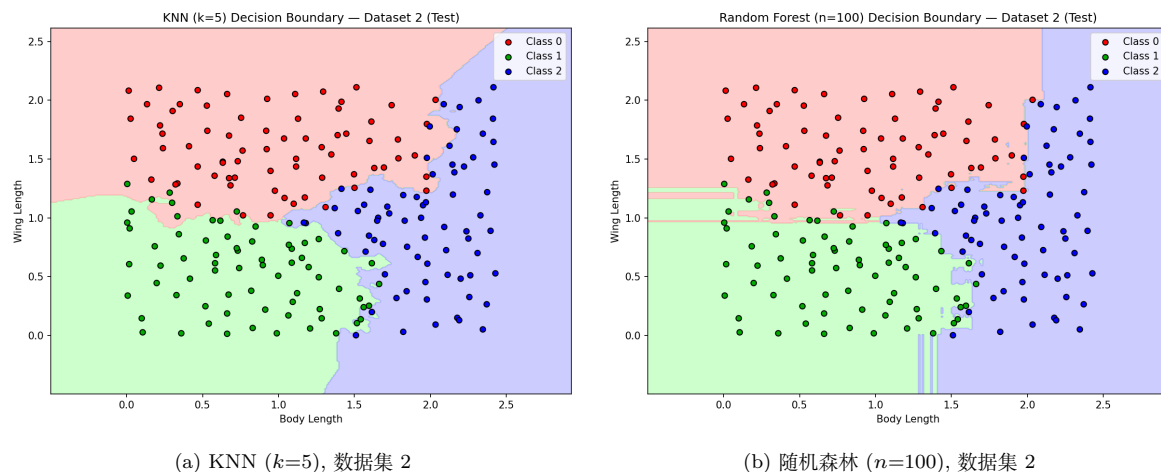


图 7: KNN 与随机森林在数据集 2 (含噪) 上的决策边界

对比图 6 与图 3 可以看出三者决策边界的本质差异:

- **神经网络**的决策边界是平滑的曲线, 因为多层感知机学习的是连续的非线性函数;
- **KNN** 的边界呈锯齿状, 由局部投票产生, 对局部数据分布敏感;
- **随机森林**的边界由分段常数区域组成, 因为决策树本质上是空间的轴平行划分。

在数据集 2 (图 7) 上, KNN 和随机森林的边界虽然也受到噪声干扰, 但整体区域划分仍然合理, 这与它们在含噪数据上优于神经网络的结论一致。

## 8 总结与讨论

### 8.1 实验结论

- 在干净数据 (数据集 1) 上, 双隐藏层 [64, 32] 配合 Leaky ReLU 激活函数可达到 100% 的分类准确率, 优于 KNN (98.10%) 和随机森林 (96.67%)。
- 含噪数据 (数据集 2) 的准确率约为 91–94%, 标签噪声是主要的性能瓶颈。意外的是, KNN ( $k=1$ ) 以 96.67%、随机森林以 96.19% 显著优于神经网络 (93.81%)。
- 激活函数对性能有显著影响: Leaky ReLU > Tanh > Sigmoid > ReLU。
- Dropout 在此简单任务上未起到正面作用, 反而降低了准确率。
- 在含噪数据上, 较小的网络 (如单层 16 神经元) 泛化性能不亚于大型网络, 说明简单模型具有更好的抗噪能力。
- 当数据标注质量无法保证时, KNN 和随机森林等传统方法可能是比神经网络更稳健的选择——它们对标签噪声有天然的容错性。

### 8.2 AI 使用说明

本实验中, 代码框架和超参数对比脚本使用了 AI (Claude Code) 辅助生成。

## 附录：环境配置

实验运行环境如下：

- **Python:** 3.x (通过 uv 管理虚拟环境)
- **PyTorch:** 2.8.0 (CPU 版本)
- **NumPy:** 2.0.2
- **Matplotlib:** 3.9.4
- **Scikit-learn:** 1.6.1

完整的依赖文件见 `requirements.txt`，可使用以下命令安装：

```
1 pip install -r requirements.txt
```